



Introdução a Técnicas de Programação

Algoritmos

Prof. André Campos
DIMAp/UFRN

O que são algoritmos?

Uma forma de especificar uma **sequência finita de passos** para se chegar a um objetivo.

Algoritmos existem há séculos (milênios) e servem para definir o que se fazer, passo a passo, de forma sistemática, para realizar uma tarefa.

Ex: Algoritmo de Euclides para divisão de números inteiros



Muhammad ibn Musa al-Khwarizmi (780-847)

*Matemático persa, “pai da álgebra”, que apresentou várias **soluções sistemáticas** para problemas matemáticos. Seu nome deu origem aos termos **Algarismo** e **Algoritmo**.*

Representações de um algoritmo

Ambiguidade



Formalismo

1. Linguagem natural
2. Fluxograma
3. Pseudocódigo
4. Linguagem de programação
 - a. Visual
 - b. Textual

Linguagem natural

Pros:

- É fácil e natural
- Serve para comunicar em alto nível de abstração (ideias)

Cons:

- É ambígua
- Não serve para descrever detalhes



[Why Operating Procedure are important](#)



Linguagem natural

- Receba dois números e diga qual o maior deles, ou então diga que eles são iguais.

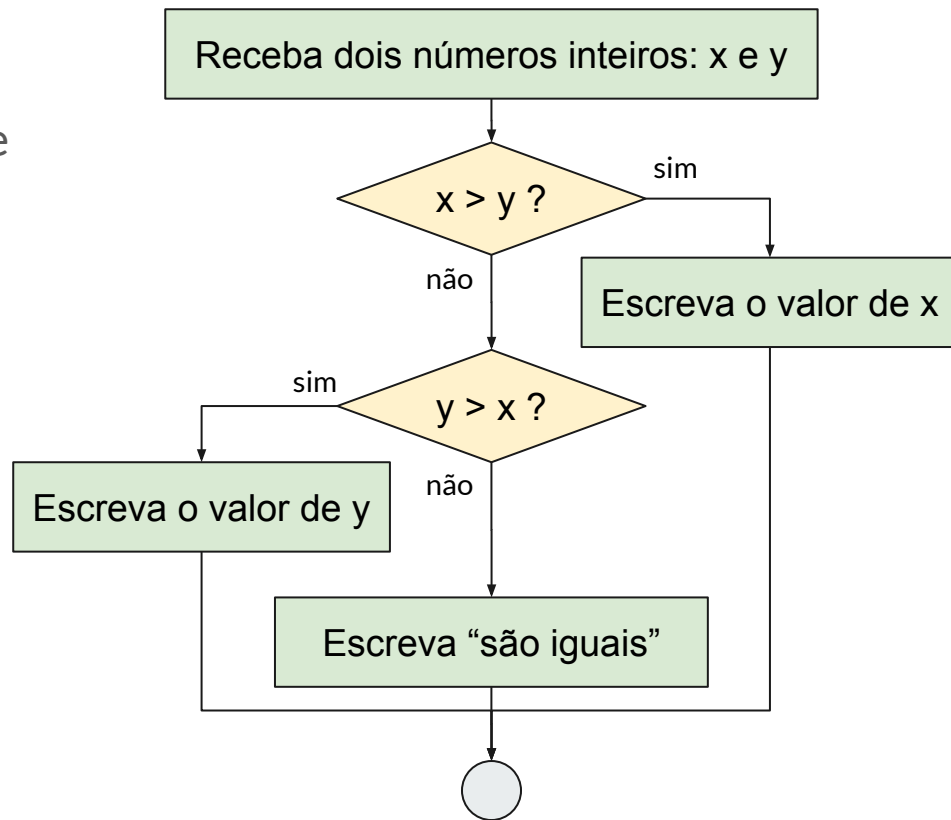
Fluxograma (diagrama de fluxo)

Pros:

- Vale o ditado: “uma imagem vale mais que 1000 palavras”
- Dá uma ideia geral de um processo

Cons:

- Não representa bem dados
- Serve apenas para processos simples/pequenos





Pseudocódigo (1)

Pros:

- Representa dados e ações
- Descreve tanto ideias abstratas quanto ações concretas

Cons:

- Não há um padrão
- Como não segue um padrão, ainda pode gerar ambiguidade

Sejam X e Y dois números inteiros
Leia um valor inteiro e armazene em X
Leia um valor inteiro e armazene em Y
Se $X > Y$, então:
 Escreva o valor de X
Senão, se $Y > X$ então:
 Escreva o valor de Y
Senão:
 Escreva "são iguais"



Pseudocódigo (2)

Pros:

- Representa dados e ações
- Descreve tanto ideias abstratas quanto ações concretas

Cons:

- Não há um padrão
- Como não segue um padrão, ainda pode gerar ambiguidade

X: número inteiro

Y: número inteiro

X ← lê um valor inteiro

Y ← lê um valor inteiro

se X > Y então:

 escreva (x)

senão se Y > X então:

 escreva (y)

senão:

 escreva ("são iguais")

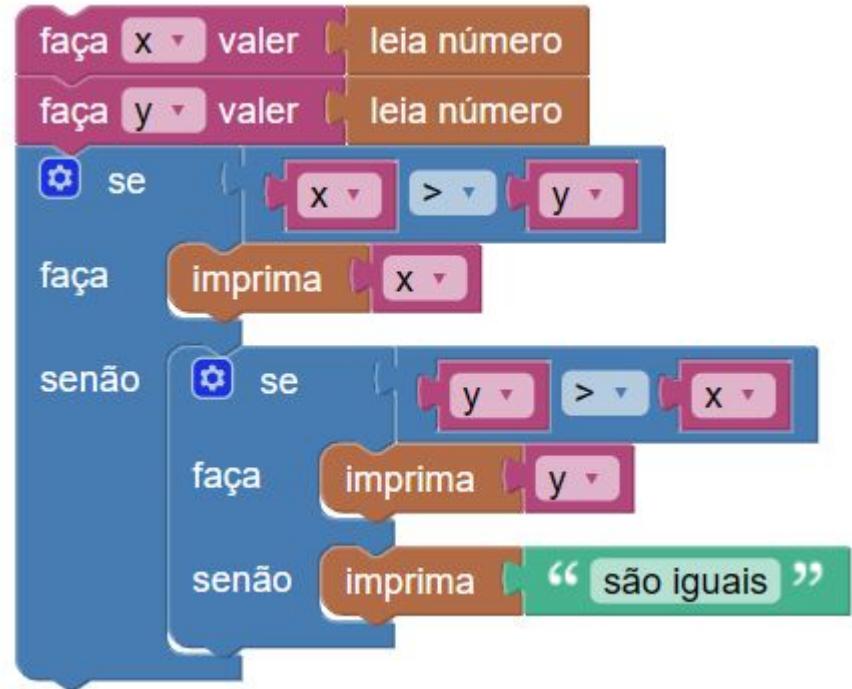
Linguagem de programação (visual)

Pros:

- Representa dados e ações
- Evita ambiguidade
- Evita erros de sintaxe

Cons:

- Muitas vezes, digitar é mais rápido que “clicar, arrastar, soltar, configurar...”





Linguagem de programação (textual)

Pros:

- Representa dados e ações
- Descreve bem ações concretas
- Não possui ambiguidade

Cons:

- Requer seguir um formalismo sintático (linguagem)

```
#include <iostream>
using namespace std;
int main() {
    int x, y;
    cin >> x >> y;
    if (x > y) {
        cout << "x é maior" << endl;
    }
    else {
        if (y > x) {
            cout << "y é maior" << endl;
        }
        else {
            cout << "x e y são iguais" << endl;
        }
    }
}
```



Problema do número espelhado

Escreva um algoritmo que recebe como entrada um valor inteiro com 3 dígitos (entre 100 e 999, inclusive) e envia para a saída seu valor espelhado.

Exemplos

Entrada	Saída
836	638
192	291
100	001



Problema do número espelhado

Solução:

1. leia e guarde um número inteiro
2. escreva o resto da divisão dele por 10
3. substitua o valor guardado pela parte inteira de sua divisão por 10
4. escreva o resto da divisão do valor guardado por 10
5. substitua o valor guardado pela parte inteira de sua divisão por 10
6. escreva o valor guardado

Entrada: 672

1. tem 672
2. escreve **2**
3. tem 67
4. escreve **7**
5. tem 6
6. escreve **6**

Saída: 276



O que um algoritmo precisa?

- **Completo:** Todas as ações precisam ser bem definidas e devem ser únicas. Ex: não pode haver “escreva A” se não houver uma ação para escrever.
- **Sem redundância:** Um conjunto de instruções só pode ter uma única forma de ser interpretada. Ex: quem ler sabe o que “escreva A” faz!
- **Determinístico:** Se as instruções forem executadas, o resultado esperado será sempre atingido. Ex: “Talvez escreva A” (não!)
- **Finito:** As instruções precisam terminar após um número limitado de passos. Ex: não pode haver “escreva A o tempo todo!”



Conceitos gerais

Dados

Variáveis

Tipos de dados:

- Valores únicos
- Sequências

Estruturas de dados:

- Listas, filas, pilhas....

Fluxo de ações

Comandos

Operadores

Expressões

Condicionais

Repetições

Organização

Procedimentos

Funções

Exemplo

Variáveis

Tipo de dado

Sejam X e Y dois números inteiros

Leia um valor inteiro e armazene em X

Leia ~~um valor inteiro e armazene em~~ Y

Se $X > Y$, então:

Escreva o valor de X

Senão, se $Y > X$ então:

Escreva o valor de Y

Senão:

Escreva "são iguais"

Comandos

Expressões



Variáveis, tipos de dados, operadores

Variáveis

Toda informação necessária para o algoritmo funcionar precisa ser **guardada em algum lugar**.

E para depois acessarmos essa informação, precisamos de um rótulo que a identifique, ou seja **identificador**.

Podemos alterar o que as “caixas” guardam durante a execução do algoritmo (ou seja, seus valores). Por isso, elas se chamam **variáveis**.



Tipo de variáveis

Os valores a serem usados nos algoritmos podem ter tipos diferentes porque os valores que podem assumir e suas **operações** são diferentes.

Se 🤯 for melhor que ☔ então
faça X ser $(\text{🕒} + \text{🏰}) / \text{🙈}$

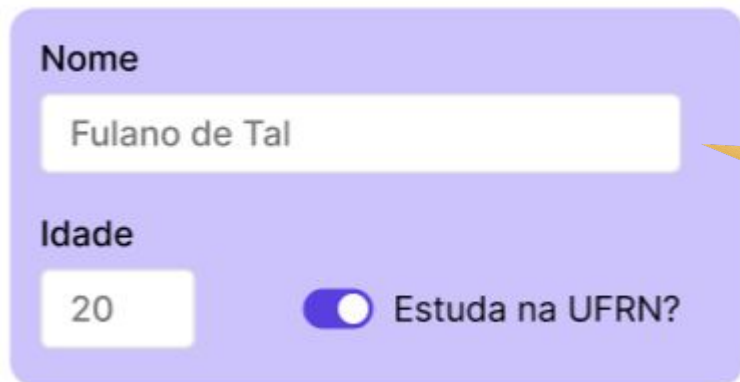
Faz sentido comparar 🤯 com ☔ ?

Faz sentido somar 🕒 com 🏰 e depois dividir por 🙈 ?

Tipo de variáveis

Os tipos das variáveis (identificadores):

- Dependem dos valores que elas armazenam
- Definem os possíveis valores que as variáveis podem representar
- Definem as possíveis operações sobre seus valores



Nome

Fulano de Tal

Idade

20

☒ Estuda na UFRN?





Tipos normalmente usados em algoritmos

Tipos básicos

- Valores numéricos
 - Valores inteiros. Ex: 67, -12, 4...
 - Valores reais. Ex: 3.4, 5.0, ...
- Valores literais (ou strings)
 - Ex: “Fulano”, “Computação”,...
- Valores lógicos
 - Falso, Verdadeiro
- Sequências
 - de tipos básicos

Operações

- Valores numéricos
 - +, -, /, *, %, ...
- Valores literais (ou strings)
 - junta valores, busca partes,...
- Valores lógicos
 - Conjunção (E) e disjunção (OU)



Operadores aritméticos

Opera sobre
valores numéricos

$p/x = 4$

Operação	Operador ¹	Exemplo	Resultado
Soma	+	5 + 8	13
Subtração	-	x - 4	0
Multiplicação	*	2 * x	8
Divisão	/	x / 2	2
Resto da divisão	% ou mod	13 % 2	1
Divisão inteira	//	13 // 2	6
Exponenciação	** ou ^	2 ^ x	16

¹ O operador depende muito da linguagem



Operadores relacionais

Opera sobre valores do mesmo tipo

Resulta em um valor lógico (Falso ou Verdadeiro)

p/ x = 4

Operação	Operador ¹	Exemplo	Resultado ²
É igual?	==	Falso==Falso	Verdadeiro
É diferente?	!= ou <>	x != 4	Falso
É maior?	>	x > 2	Verdadeiro
É menor?	<	x < 2	Falso
É maior ou igual?	>=	x >= 20	Falso
É menor ou igual?	<=	x <= x	Verdadeiro

¹ O operador depende muito da linguagem



Operadores lógicos

Opera sobre valores lógicos

Resulta em um valor lógico

p/ x = 4

Operação	Operador ¹	Exemplo	Resultado ²
E (conjunção)	and ou &&	x >= 4 && x < 2	Falso
OU (disjunção)	or ou	x >= 4 x < 2	Verdadeiro
NÃO (negação)	not ou !	!(x >= 4 x < 2)	Falso

¹ O operador depende muito da linguagem



Problema do número espelhado

Escreva um algoritmo que recebe como entrada um valor inteiro com 3 dígitos (entre 100 e 999, inclusive) e envia para a saída seu valor espelhado.

Além dos operadores, você pode usar expressões como:

- “leia” para receber o valor de entrada
- “escreva” para enviar algo para a saída
- “variável $\leftarrow$ ” para guardar o valor de uma expressão em uma variável

Exemplos

Entrada	Saída
836	638
192	291
100	001

Problema do número espelhado

Solução:

1. V : número inteiro
2. leia V
3. escreva $V \bmod 10$
4. $V \leftarrow V // 10$
5. escreva $V \bmod 10$
6. $V \leftarrow V // 10$
7. escreva V

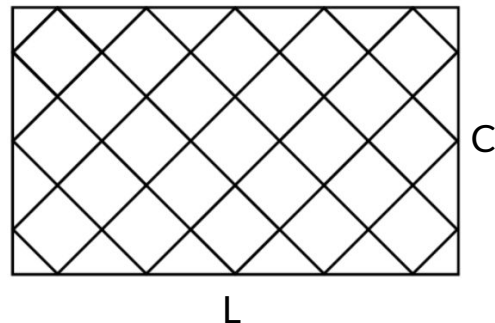
Entrada: 672

1. $V = ?$
2. $V = 672$
3. escreve 2
4. $V = 67$
5. escreve 7
6. $V = 6$
7. escreve 6

Saída: 276

Problema do piso (adaptado da OBI 2018)

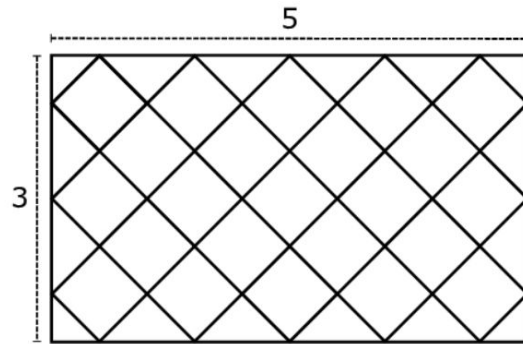
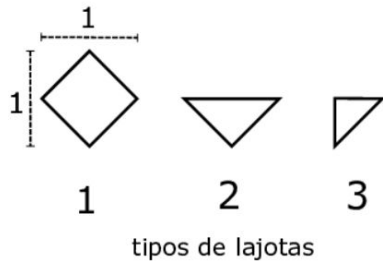
Sua antiga escola pretende trocar o piso de uma sala de aula e a diretora aproveitou a oportunidade para passar uma tarefa aos alunos. A sala tem o formato de um retângulo de largura L metros e comprimento C metros, onde L e C são números inteiros.



A diretora precisa comprar lajotas de cerâmica para cobrir todo o piso da sala. Seria fácil calcular quantas lajotas seriam necessárias se cada lajota fosse um quadrado de 1 metro de lado. O problema é que a lajota que a diretora quer comprar é um quadrado que possui 1 metro de diagonal, não de lado. Além disso, ela quer preencher o piso da sala com as diagonais das lajotas alinhadas aos lados da sala, como na figura.

Problema do piso

A tarefa que a diretora passou para os alunos é calcular o número de lajotas dos tipos 1 e 2 que serão necessárias. Na figura, para uma largura de 3m ($L=3$) e um comprimento de 5m ($C=5$), foram necessárias 23 do tipo 1 e 12 do tipo 2.



Exemplos

Para $L = 3$ e $C = 5$

$T1 = 23$

$T2 = 12$

Para $L = 1$ e $C = 1$

$T1 = 1$

$T2 = 0$



Problema do piso

Escreva um algoritmo (em **pseudocódigo**) para calcular a quantidade de lajotas do tipo 1 e do tipo 2 necessárias, dadas as dimensões da sala (L e C).

Além dos operadores, você pode usar expressões como:

- “leia” para a entrada de dados (largura e comprimento)
- “escreva” para a saída de dados (a quantidade de lajota dos tipos T1 e T2)
- “variável $\leftarrow$ ” para guardar o valor de uma expressão em uma variável



Condicionais

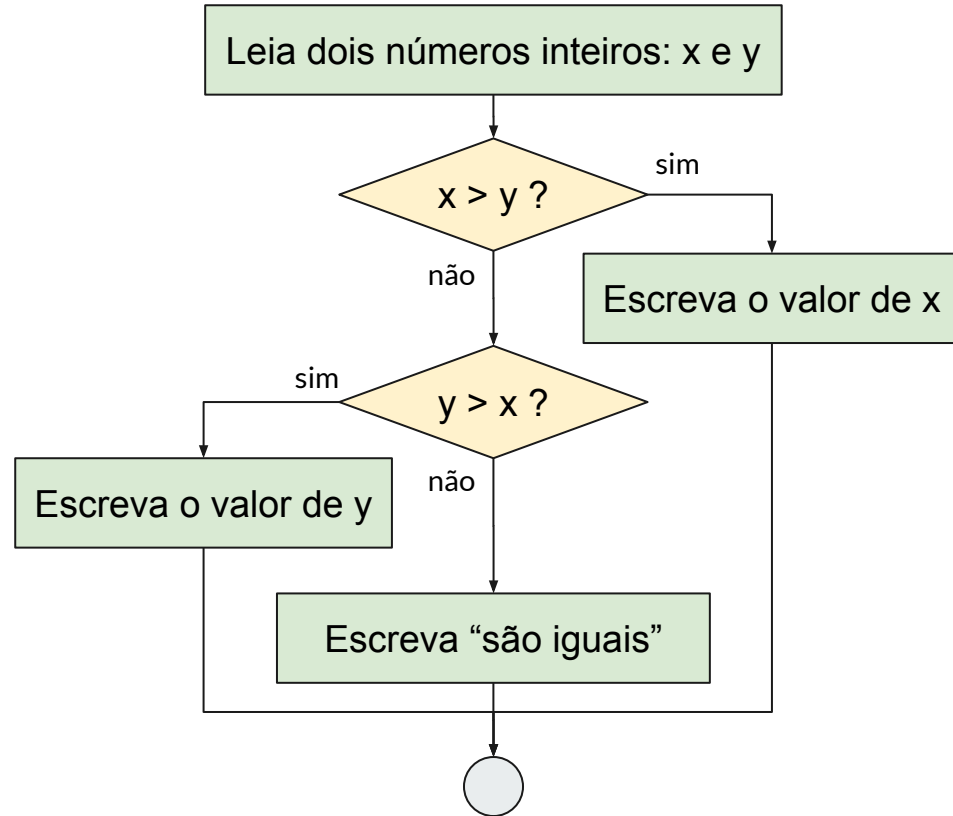
Fluxo de ação alternativo

Inúmeras vezes, precisamos definir diferentes sequências de ações.

Para escolher o fluxo, precisamos testar uma condição:

expressão lógica:

Falso ou **Verdadeiro**



Se ... então

Usa **bloco de comandos** para especificar um fluxo alternativo

valor: número inteiro

escreva "Digite um valor até 10"

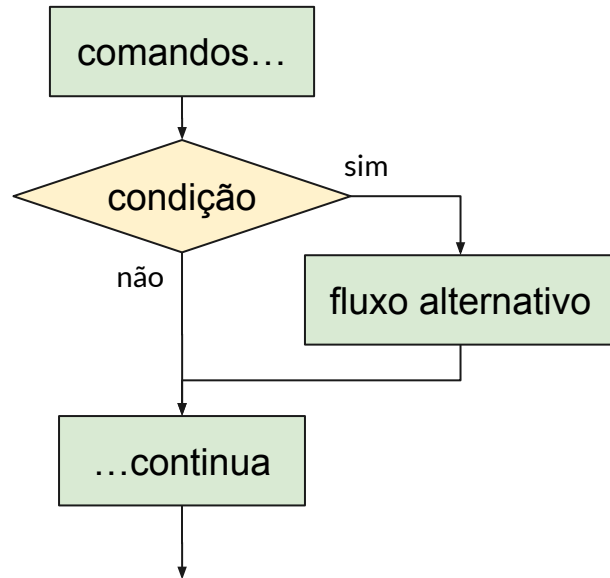
Leia valor

Se valor > 10 **então**

| escreva "Vou considerar o valor 10"

| valor $\leftarrow$ 10

Escreva "o dobro é ", $2 * \text{valor}$



Se ... senão

Usa **bloco de comandos** para especificar um fluxo alternativo

valor: número inteiro

escreva "Digite um valor até 10"

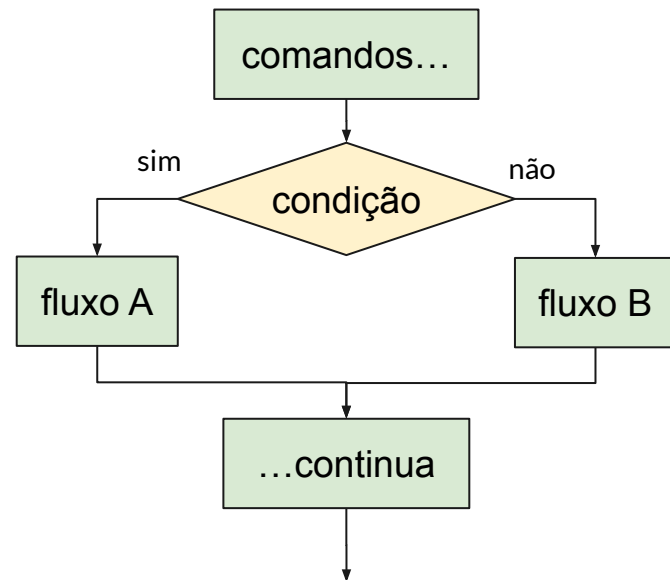
Leia valor

Se valor > 10 **então**

| escreva "Eu pedi um valor até 10!!!"

Senão

| Escreva "o dobro é ", 2 * valor



Se... senão pode ser encadeado (se... senão se... senão...)



Problema: conversor de temperatura

Apresente um algoritmo (em pseudocódigo) que, dado o valor de uma temperatura em uma escala (Celsius, Kelvin ou Fahrenheit), informe o seu equivalente em outra escala.

Seu algoritmo recebe 3 informações:

- O valor da temperatura,
- A escala original, e
- A escala desejada (para a qual o valor será transformado).

Leve em conta que: $K = C + 273,15$

$$F = 1,8 \times C + 32$$

Problema: conversor de temperatura

Possível solução

temperatura : número real
original, desejada : literais

leia (temperatura)

leia (original)

leia (desejada)

```
se original = "Celsius" então
|   se desejada = "Kelvin" então
|       escreva temperatura + 273.15
|   senão se desejada = "Fahrenheit" então
|       escreva 1.8 * original + 32
senão se original = "Kelvin" então
|   se desejada = "Celsius" então
|       escreva temperatura - 273.15
|   senão se desejada = "Fahrenheit" então
|       ...
```



Repetições



Problema do número espelhado

Como faria para resolver o problema do número espelhado se a entrada fosse um valor inteiro qualquer?

Exemplos

Entrada	Saída
82	28
1920991	1990291
1001	1001

Repetições

Muitas vezes, precisamos repetir procedimentos (padrões que se repetem)

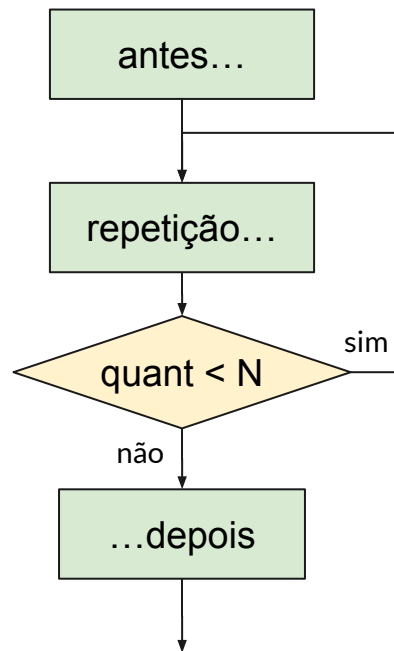
Possui várias formas:

comandos antes da repetição

Repita N vezes :

| comandos repetidos

comando depois da repetição



Repetições

Muitas vezes, precisamos repetir procedimentos (padrões que se repetem)

Possui várias formas:

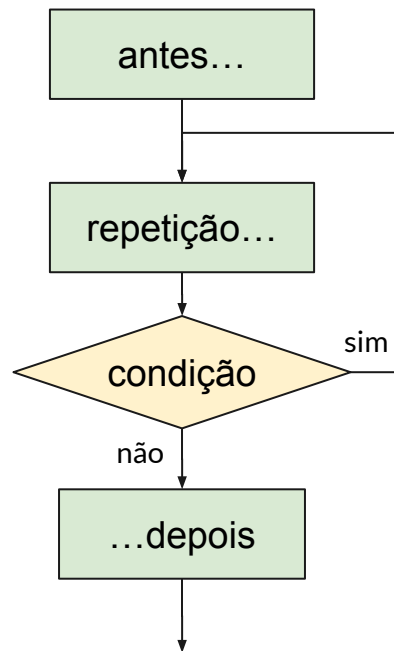
comandos antes da repetição

faça:

| comandos repetidos

enquanto (condição)

comando depois da repetição

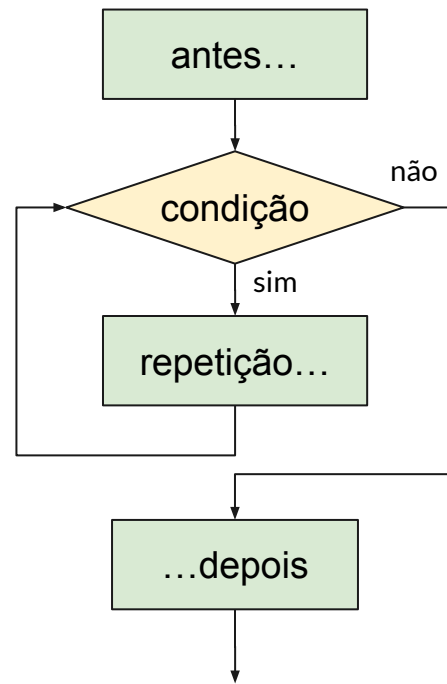


Repetições

Muitas vezes, precisamos repetir procedimentos (padrões que se repetem)

Possui várias formas:

comandos antes da repetição
enquanto (condição) **faça**:
| comandos repetidos
comando depois da repetição





Repetições

Há formas de repetições para percorrer as sequências

Possui várias formas:

comandos antes da repetição

para cada elemento da sequência **faça**:

| comandos com o elemento

comando depois da repetição



Problema do número espelhado

Escreva um algoritmo que recebe como entrada um valor inteiro (qualquer) e envia para a saída seu valor espelhado.

Exemplos

Entrada	Saída
82	28
1920991	1990291
1001	1001



Problema do número espelhado

Possível solução:

```
V: número inteiro  
leia V  
enquanto V  $\neq$  0 faça  
|   escreva V mod 10  
|    $V \leftarrow V // 10$ 
```

Exemplos

Entrada	Saída
82	28
1920991	1990291
1001	1001